STOCK MANAGEMENT SYSTEM

SYSTEM DOCUMENTATION

SUBMITTED BY,

LANCE KIPROP

DIT-01-0309/2024

A SYSTEM DOCUMENTATION SUBMITTED IN PARTIAL FULFILMENT FOR THE AWARD OF DIPLOMA IN INFORMATION TECHNOLOGY BY ZETECH UNIVERSITY

APRIL 2025

DECLARATION

I, Lance Kiprop, hereby declare that this project titled ‘Stock Management System’ is my original work and has been carried out by me under the guidance of both my lecturer and the supervisor. I confirm that all sources used in this project have been properly acknowledged and cited where necessary.

I further declare that this project has not been submitted, in part or in full, for any other academic award or qualification at this or any other institution.

Signed: _____
Date: _____

DEDICATION

I dedicate this project to my parents, for their endless support, guidance and encouragement throughout my studies.

I also dedicate this work to my lecturers and classmates, who have contributed to my learning journey through their guidance, advice and motivation.

Finally, I dedicate this project to anyone who aspires to develop innovative systems and technological solutions as a contribution to the field of information technology.

ABSTRACT

This project presents a Stock Management System, a web-based application designed to simplify the management of product inventory in a small clothing shop. The system was developed using PHP for server-side logic, MySQL for the database and HTML/CSS for the user interface. Validation checks ensure secure login, proper data entry and controlled access to system dashboards. Testing was conducted to ensure that all features function correctly and corrections were made to improve system reliability. The system was deployed using the XAMPP server allowing access through a web browser. The project provides an efficient, secure and user-friendly platform for managing inventory, tracking sales and monitoring stock levels.

DEFINITION OF KEY TERMS

- **Sales Page**
  A web page in the system that allows users to record product sales, calculate total price, and update stock levels in the database.
- **Stock Management**
  The process of monitoring and controlling product inventory to ensure that available stock is accurate, prevent overselling, and manage replenishment.
- **Validation**
  A process in programming that checks if user inputs are correct, complete, and within allowed limits before processing them in the system.
- **Inventory**
  A list of all products available in the system, including their quantities, prices, and other details needed for sales and stock management.
- **Product**
  An item or good that is stored in the system inventory and available for sale. Each product has details such as name, price, and quantity in stock.
- **Sales Transaction**
  A recorded operation in the system where a product is sold to a customer. The transaction updates the database and reduces the available stock.
- **Database**
  A structured collection of data stored electronically that allows the system to store, manage and retrieve information such as products, users and sales records.
- **Dashboard**
  The main interface of the system where users can view summaries such as stock levels, sales information and system alerts.
- **Administrator (Admin)**
  A system user with full privileges who manages products, monitors stock levels, and controls system operations.
- **User Interface (UI)**
  The visual part of the system that allows users to interact with the application through pages, forms and buttons.
- **Transaction Record**
  A stored entry in the database that keeps details of a completed activity such as a product sale or stock update.
- **Stock Level**
  The quantity of a particular product available in the system inventory at a given time.
- **Database Table**
  A structured arrangement of data in rows and columns used to store related information within a database.
- **Server**
  A computer or software environment that hosts the system and allows users to access it through a web browser.

ABBREVIATIONS AND ACRONYMS

- **HTML** – HyperText Markup Language
- **JS** – JavaScript
- **POS** – Point of Sale
- **CRUD** – Create, Read, Update and Delete (basic database operations)
- **API** – Application Programming Interface
- **IDE** – Integrated Development Environment
- **HTTP** –Hypertext Transfer Protocol
- **XAMPP** –Cross-Platform Apache, MySQL, PHP and Perl
- **URL** – Uniform Resource Locator
- **DB** – Database
- **ERD** – Entity Relationship Diagram
- **PHP** – Hypertext Preprocessor
- **MVC** – Model View Controller
- **SQL** – Structured Query Language
- **DBMS** – Database Management System
- **UI** – User Interface
- **LAN** – Local Area Network
- **SSL** – Secure Sockets Layer
- **HTTPS** – HyperText Transfer Protocol Secure
- **OS** – Operating System
- **RAM** – Random Access Memory
- **GUI** – Graphical User Interface
- **POS** – Point of Sale System
- **CSS** – Cascading Style Sheets

LIST OF FIGURES

Fig 2.2.1 Login Page Design
Fig 2.2.2 Dashboard Page Design
Fig 2.2.3 Products Page Design
Fig 2.2.4 Sales Page Design
Fig 2.3.1 Login Process Flowchart
Fig 2.3.2 Sales Processing Flowchart
Fig 2.3.3 System Update Flowchart
Fig 3.2.1 Login Page Interface
Fig 3.2.2 Sales Page Interface
Fig 3.2.3 Admin Dashboard Interface
Fig 3.2.4 Stock Management Page Interface
Fig 3.3.1 Login Validation Code
Fig 3.3.2 Admin Dashboard Validation Code
Fig 3.3.3 Stock Management Validation Code
Fig 3.3.4 Sales Validation Code

LIST OF TABLES

Table 1.4 Functional Requirements Table
Table 1.6 Project Schedule Breakdown
Table 3.3.5.1 Users Table Structure
Table 3.3.5.2 Products Table Structure
Table 3.3.5.3 Sales Table Structure

TABLE OF CONTENTS

Contents

DECLARATION 2

DEDICATION 3

ABSTRACT 4

DEFINITION OF KEY TERMS 5

ABBREVIATIONS AND ACRONYMS i

LIST OF FIGURES ii

LIST OF TABLES iii

TABLE OF CONTENTS iv

CHAPTER ONE: PROJECT PLANNING AND ANALYSIS 1

(WORKPLAN) 1

1.1 Statement of Problem 1

1.2 Study Justification 1

1.3 System Objectives 1

1.3.1 General Objective 1

1.3.2 Specific Objectives 1

1.4 Functional Requirements 2

1.5 Breakdown of Tools & Resources to Be Used 2

1. Software Tools 2

2. Development Environment 2

3. Deployment Resources 3

4. Hardware Resources 3

5. Human Resources 3

1.6 Project Schedule Breakdown 1

CHAPTER TWO: DESIGN AND MODELING 2

2.1 Introduction to Modelling 2

2.2 User Interface Models 2

2.2.1 Login Page Design 2

2.2.3 Products Page Design 4

2.2.4 Sales Page Design 5

2.3 Logic Models 6

2.3.1 Login Process Flowchart 6

2.3.2 Sales Processing Flowchart 7
2.3.3 System Update Flowchart 9
CHAPTER THREE: SYSTEM IMPLEMENTATION 11
(DEVELOPMENT, TESTING AND DEPLOYMENT) 11
3.1 Introduction 11
3.2 User Interface Development 11
3.2.1 Login Page Development 11
3.2.2 Sales Page Development 14
3.2.4 Admin Dashboard Development 15
3.2.5 Stock Management Page Development 17
3.3 Logic Development 18
3.3.1 Login Validation Code 18
3.3.2 Admin Dashboard Validation Code 19
3.3.3 Stock Management Validation Code 19
3.3.4 Sales Validation Code 20
3.3.5 Database Design 20
3.4 Testing 22
3.5 Deployment 22
CHAPTER FOUR: CONCLUSION AND RECOMMENDATION 24
4.1 Conclusion 24
4.2 Recommendation 24
REFERENCES 25

CHAPTER ONE: PROJECT PLANNING AND ANALYSIS

(WORKPLAN)

1.1 Statement of Problem

A small clothing shop in a local market uses a manual notebook to record sales and manage stock. This method makes it difficult for the shop owner to accurately monitor inventory levels, identify the most popular clothing items and understand seasonal sales trends. As a result, the business often faces challenges such as stock shortages, overstocking of slow-moving items, and poor decision making regarding purchases. These problems reduce efficiency and may lead to financial losses. There is therefore a need for a better stock management approach that can improve record keeping and support informed business decisions.

1.2 Study Justification

This study is important because proper stock management plays a key role in the success of small retail businesses. By addressing the challenges associated with manual record keeping, the study aims to highlight the benefits of an improved stock management system. The findings of this study will help the shop owner manage inventory more effectively, save time, reduce errors and understand customer demand patterns. Additionally, the study may serve as a reference for other small businesses facing similar stock management challenges.

1.3 System Objectives

1.3.1 General Objective

(i) To develop an automated stock management system that improves record keeping and enhances monitoring of a clothing shop.

1.3.2 Specific Objectives

(i) To develop a system that has a feature that automatically updates stock levels after each sale or restocking.

(ii) To develop a digital system for recording daily sales transactions accurately and efficiently.

(iii) To implement an alert system that notifies the shop owner when stock levels fall below a minimum threshold.

(iv) To design a secure user login system that restricts access to authorized users only.

1.4 Functional Requirements

User	User Activities	Features
Shop Owner / Admin	- Add new clothing items - Edit product details - Record sales transactions - View sales history	- system dashboard that has add, editing and viewing stock features
Staff	- Record daily sales, Search for item availability, Print receipts.	-Point of Sale (POS) Interface, Inventory Search, Receipt Generation.

Table 1.4 Functional Requirements Table

The functional requirements describe the main activities that users can perform within the system and the corresponding features provided by the system to support those activities. The Stock Management System supports two main users: the shop owner (administrator) and the sales staff. Each user interacts with the system through specific modules designed to simplify stock monitoring, product management and sales processing.

1.5 Breakdown of Tools & Resources to Be Used

1. Software Tools

- Front-End: HTML, CSS and JavaScript – used for designing the user interface, styling web pages and creating interactive elements.
- Back-End: PHP or Python (Django/Flask) – to handle the system logic, user logins, and information retrieval.
- Database: MySQL – to securely store and organize product records, sales transactions, and user accounts in the stock management system.
- Security: SHA-256 Hashing – for protecting user passwords and improving system security.

2. Development Environment

- IDE: Visual Studio Code (VS Code) – for writing and debugging the project code.
- Local Server: XAMPP or WAMP – to run the database and the system locally on a computer during development.
- Version Control: Git & GitHub – for saving different versions of the code and tracking progress.

3. Deployment Resources

- 000webhost or Render: Free hosting services to make the system accessible online for testing.
- GitHub Pages: To host the project documentation and the static front-end design.
- Localhost: For demonstrating the system's functionality during the final presentation.

4. Hardware Resources

- Development Laptop: A computer with at least 4GB RAM to build and test the software.
- Client Devices: Smartphones and tablets used to test whether the system is responsive and accessible on different devices.
- Internet Router: To provide the connectivity needed for research and cloud database synchronization.

5. Human Resources

- System Developer: You, the student responsible for designing, coding, and testing the system.
- System Supervisor: The lecturer who provides professional guidance and reviews the project milestones.
- Pilot Users: A small group of fellow students and staff who will test the system to find any bugs or errors.

1.6 Project Schedule Breakdown

	PROJECT MILESTONES					
WEEKS	**Project Planning & Analysis** **(System Documentation: Coverpage & Chapter One)**	**Project Design & Modeling** **(System Documentation Chapter Two)**	**Project Development & Testing** **(System Documentation Chapter Three)**	**Project Deployment** **(System Documentation Chapter Three)**	**Final Touches of System Documentation** **(Preliminary Pages, Chapter Four & References)**	**Project Presentation**
5-9 May						
12-16 May						
19-23 May						
26-30 May						
2-6 June						
9-13 June						
16-20 June						
23-27 June						
30-4 July						
7-11 July						
14-18 July						
21-25 July						
11 Aug						

Table 1.6 Project Schedule Breakdown

Blue initially means that milestone has not achieved. Once you achieve that milestone, fill it with any other color of your choice to mean you're done with that part. That helps you to monitor your own progress versus the time scheduled.

When printing, this page must be printed in LANDSCAPE as the rest is printed in portrait

CHAPTER TWO: DESIGN AND MODELING

2.1 Introduction to Modelling

This chapter presents the design techniques to be used during the development of the inventory management system for the local clothing shop. Before the actual system development begins several diagrams are created to help visualize how the system will look and function so these diagrams help in understanding system requirements clearly and identifying potential problems early also ensuring that the final system meets user needs. It also serves as a guide during implementation thus reducing development errors and saving time.

2.2 User Interface Models

This section presents the user interface (UI) designs that will be sketched during system development.

2.2.1 Login Page Design

The login page provides secure access for authorized users and is designed with a simple interface to reduce errors and improve usability. The interface is kept simple to reduce errors and improve usability. It will include input fields for username and password, a login button and system branding. the validation messages will guide the user in case of incorrect credentials.

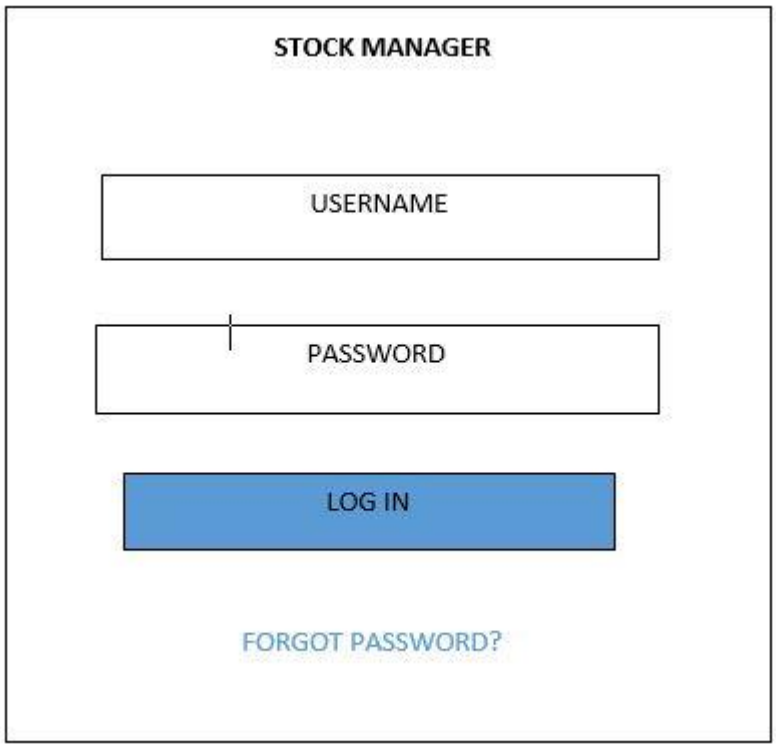


Fig 2.2.1 Login Page Design

2.2.2 Dashboard Page Design

The dashboard page will be designed to serve as the main control center of the inventory management system. It will provide users with a quick preview of important information after login. The dashboard displays summary cards such as total products, available stock, low stock alerts and total sales. It also contains navigation links that allow users to easily access other modules such as products, sales and reports. This design will help users make fast decisions without going through all pages.

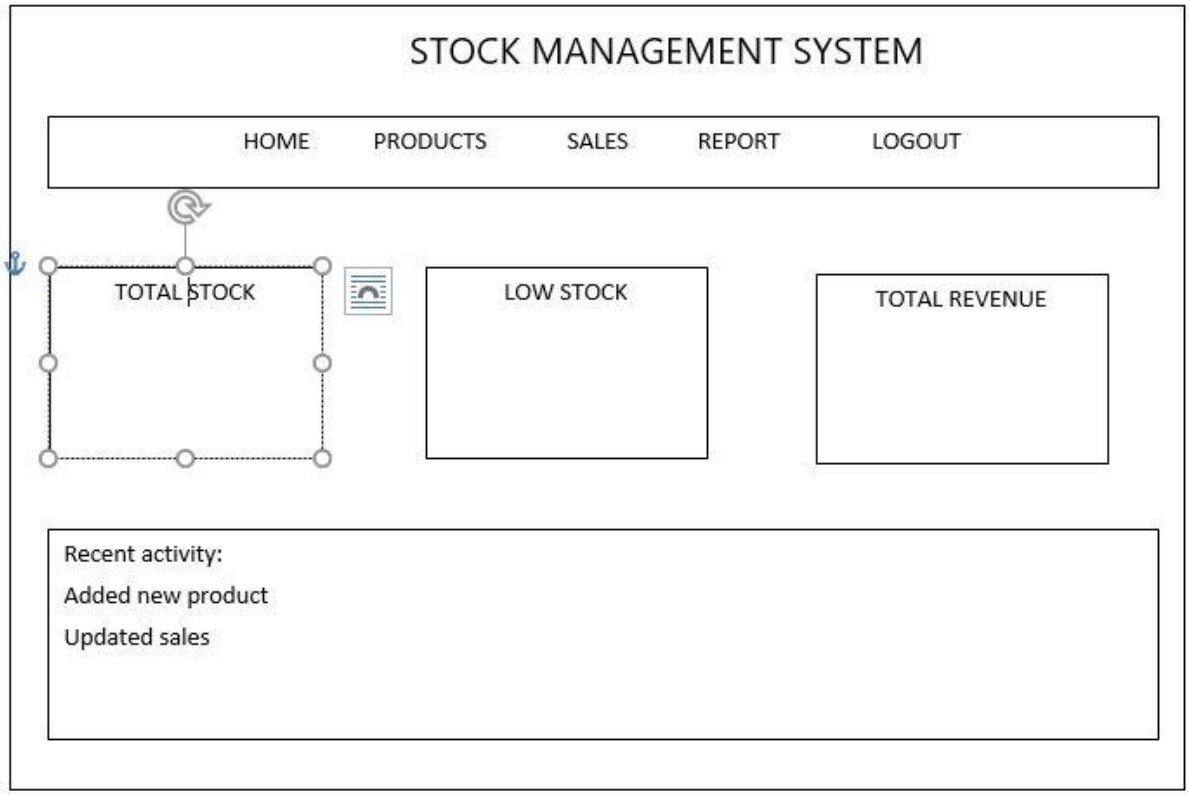


Fig 2.2.2 Dashboard Page Design

2.2.3 Products Page Design

The products page will be designed to allow the admin to manage all clothing items in the shop. This page will enable adding, editing and deleting products while displaying current stock levels. Each product includes details such as product name, category, size, color, price and quantity. The design will ensure that stock management is simple and well organized.

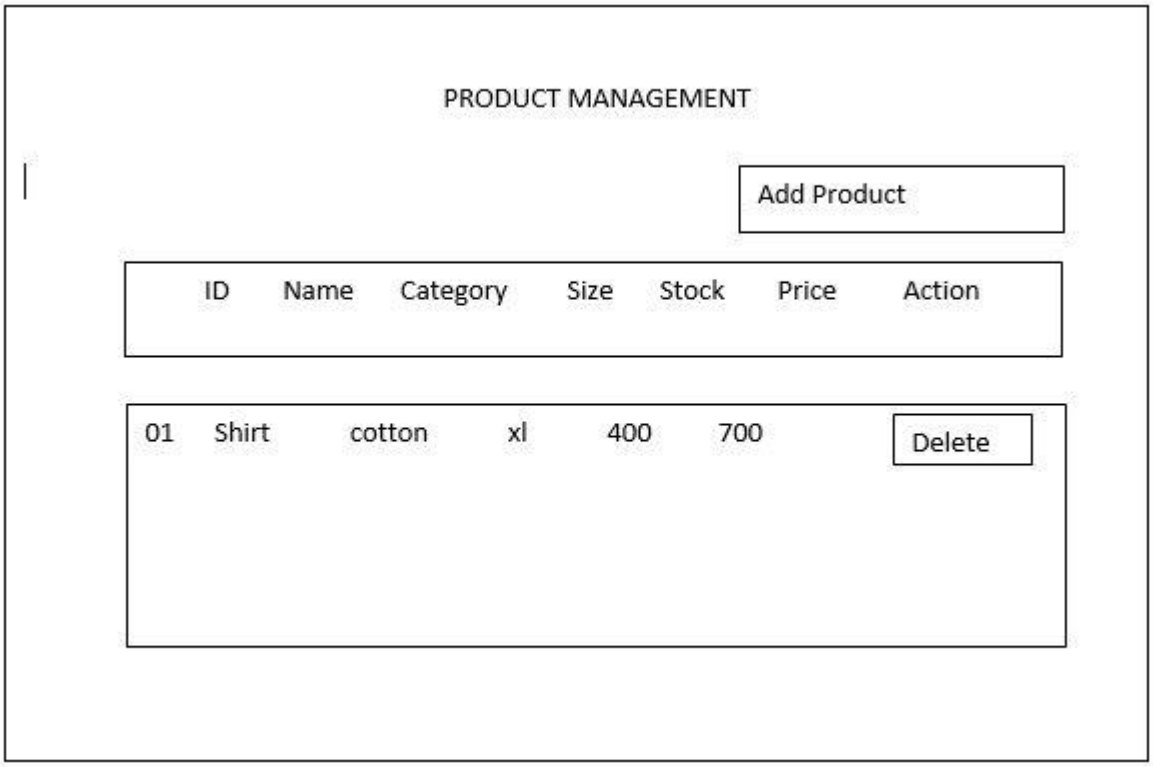


Fig 2.2.3 Products Page Design

2.2.4 Sales Page Design

The sales page will be designed to support the checkout and sales process. It allows the cashier to select products, input quantities and calculate the total cost automatically. Once a sale is confirmed the system updates the inventory in real time.

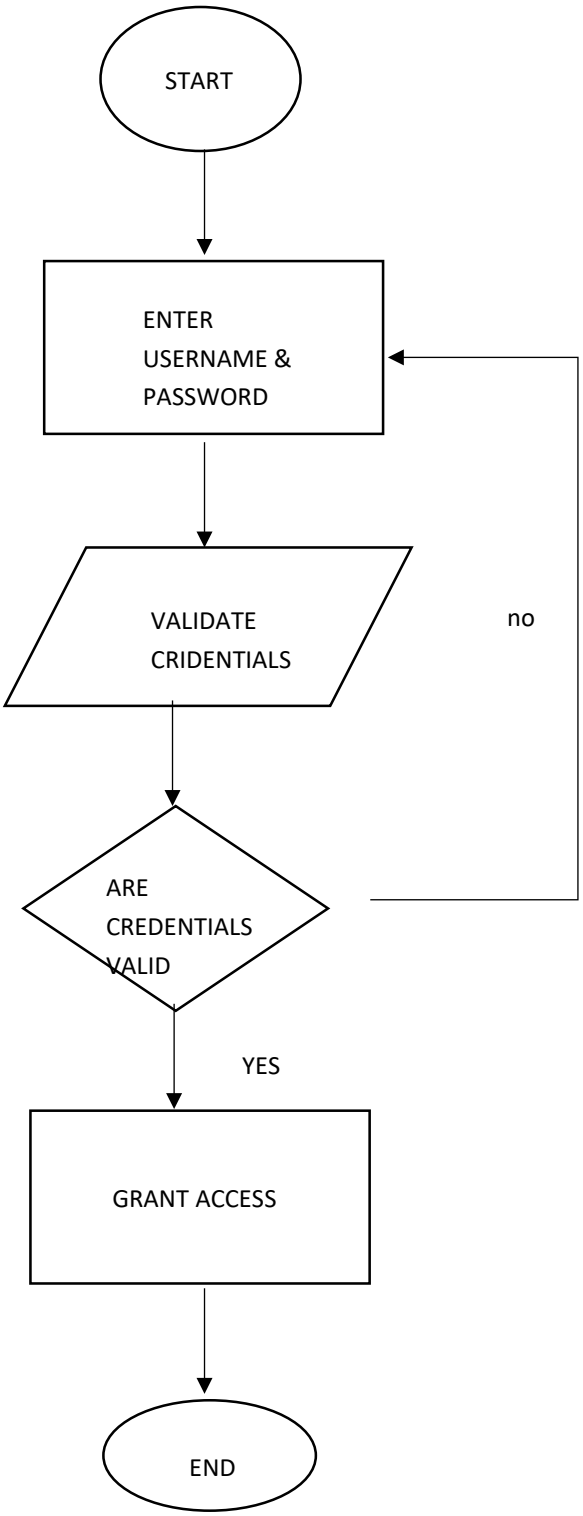
Fig 2.2.4 Sales Page Design

2.3 Logic Models

This section presents the flowcharts to be used in modelling the logical flow of processes within the Inventory Management System. These flowcharts will be created to clearly illustrate how different system operations are carried out before implementation.

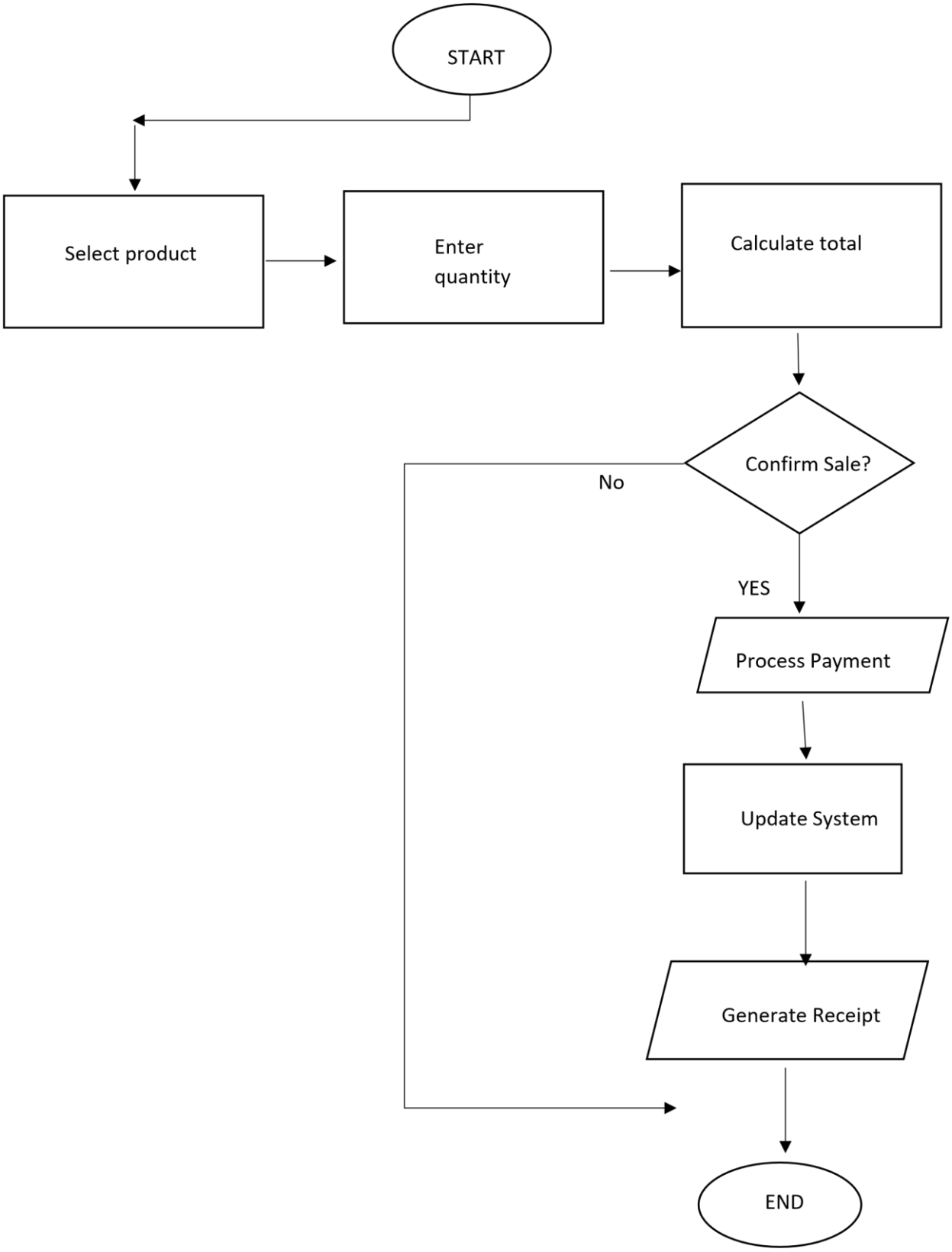
2.3.1 Login Process Flowchart

The login process flowchart illustrates the steps to be followed when a user attempts to access the system. It begins when the user enters their login credentials then the system validates the credentials. If the credentials are correct, access to the dashboard is granted otherwise an error message is displayed and the user is prompted to retry.



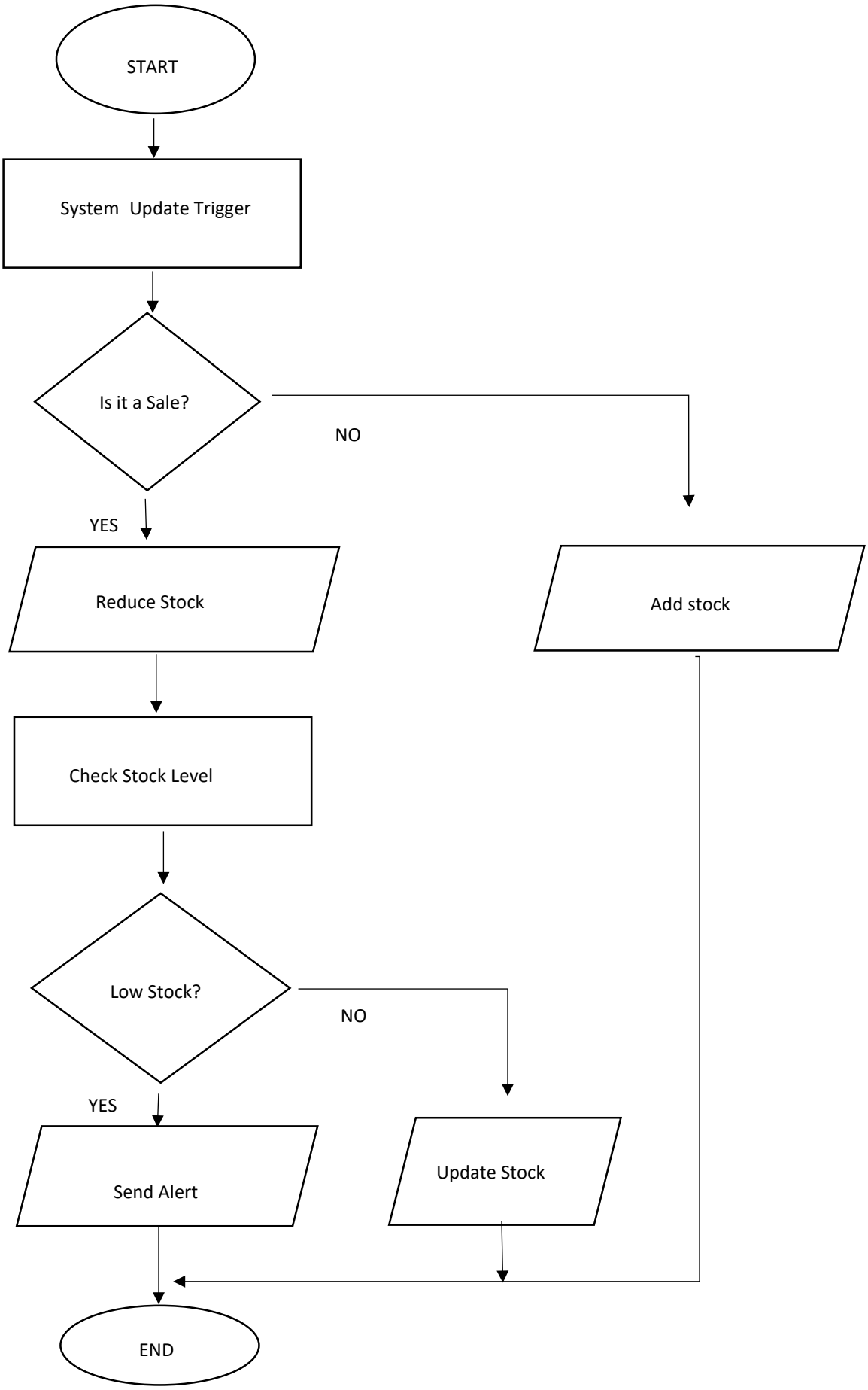
2.3.2 Sales Processing Flowchart

This flowchart describes how sales transaction is handled in the system. It starts when the cashier selects products and quantities then the system calculates the total cost it confirms payment and records the sale and updates inventory automatically.


START
Select product
Enter quantity
Calculate total
Confirm Sale?
No
YES
Process Payment
Update System
Generate Receipt
END

2.3.3 System Update Flowchart

The system update flowchart shows how stock levels will be adjusted in the system. The system can be updated through sales transactions and manual stock additions by the admin.


START
System Update Trigger
Is it a Sale?
NO
YES
Reduce Stock
Add stock
Check Stock Level
Low Stock?
NO
YES
Send Alert
Update Stock
END

CHAPTER THREE: SYSTEM IMPLEMENTATION

(DEVELOPMENT, TESTING AND DEPLOYMENT)

3.1 Introduction

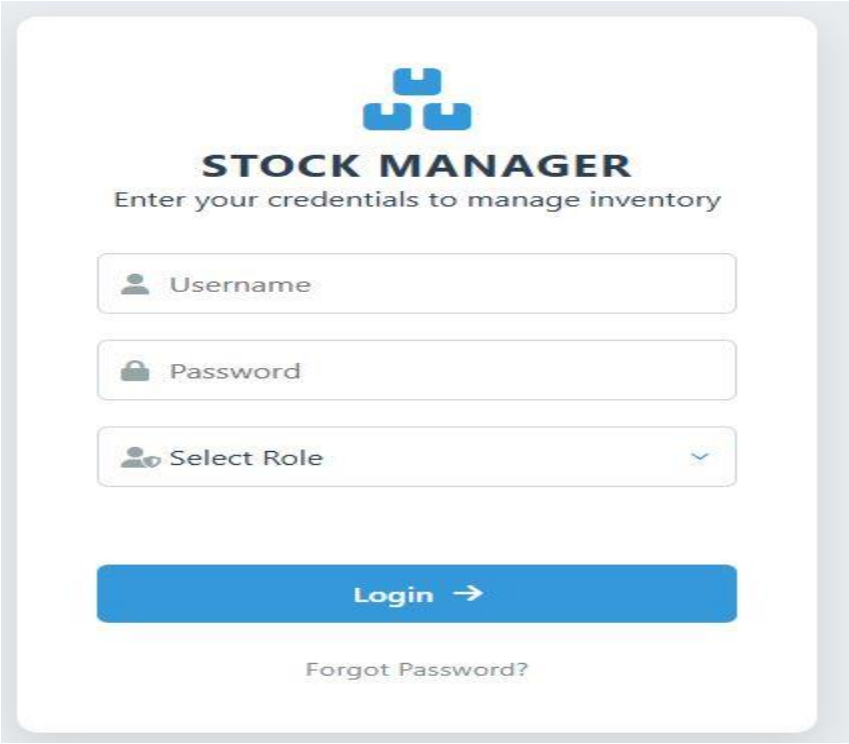
In this chapter I describe the process I followed to develop, test, and deploy the Stock Management System. I took several steps from setting up the development environment to building the different components of the system. During the development process, I installed Visual Studio Code which I used in my front end development which holds the structure of the website which displays the details and also xampp. These are the software that I used in my web development to design and implement the system interface and functionality.

I used XAMPP to create and implement the database which stores system information such as user accounts, product records and sales transactions. After developing the system, I carried out testing to ensure that the different features work correctly and that the system performs as expected. Finally, the system was deployed so that administrators and staff can access it through a web browser to manage products and record sales transactions. This chapter therefore presents the practical steps and tools used in implementing the Stock Management System.

3.2 User Interface Development

3.2.1 Login Page Development

This is the page where the users will enter their credentials where they will be proven in order to be granted access to their respective dashboards according to their role If the user is an administrator, access is granted to the admin dashboard where stock and sales can be managed.



Login page

```
<head>
</head>
<body>

<div class="login-container">
    <div class="login-header">
        <i class="fas fa-boxes-stacked fa-3x"></i>
        <h2>STOCK MANAGER</h2>
        <p>Enter your credentials to manage inventory</p>
    </div>

    <form id="loginForm" method="POST" action="login.php">
        <div class="input-group">
            <i class="fas fa-user"></i>
            <input type="text" name="username" id="username" placeholder="Username" required autocomplete="username">
        </div>

        <div class="input-group">
            <i class="fas fa-lock"></i>
            <input type="password" name="password" id="password" placeholder="Password" required autocomplete="current-password">
        </div>

        <div class="input-group">
            <i class="fas fa-user-shield"></i>
            <select name="role" id="role" required>
                <option value="" disabled selected>Select Role</option>
                <option value="staff">Staff</option>
                <option value="admin">Admin</option>
            </select>
        </div>
```

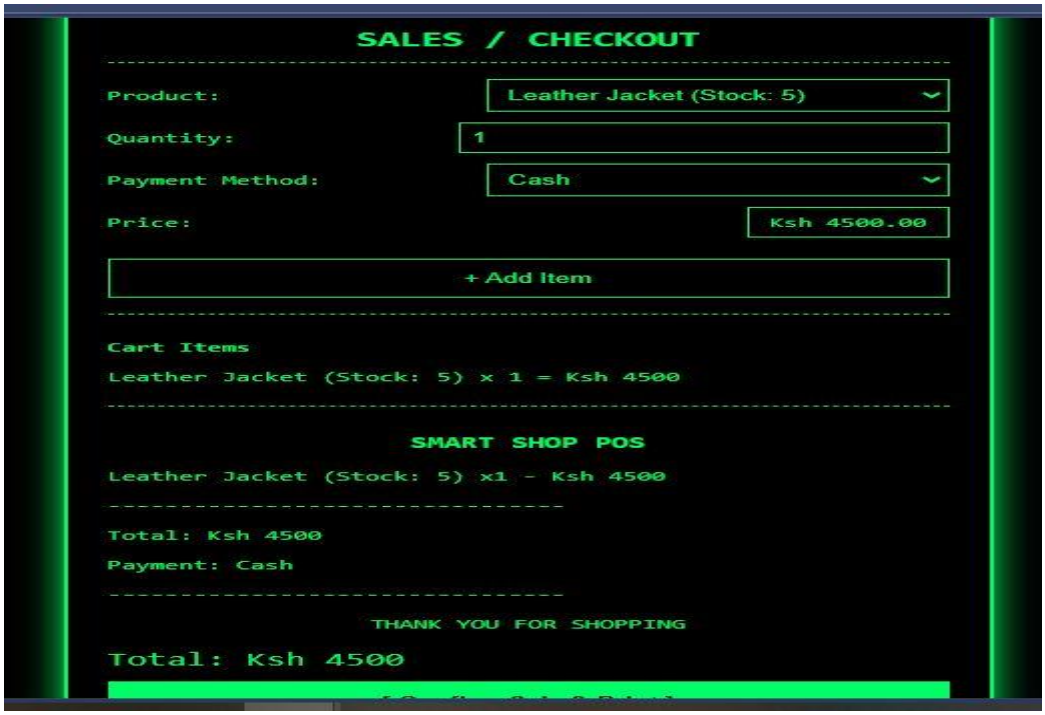
Screenshot of its html

```
<style>

    * {
        margin: 0;
        padding: 0;
        box-sizing: border-box;
        font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    }
    body {
        background: linear-gradient(135deg, #2c3e50 0%, #34495e 100%);
        height: 100vh;
        display: flex;
        justify-content: center;
        align-items: center;
    }
    .login-container {
        background: #ffffff;
        width: 100%;
        max-width: 400px;
        padding: 40px;
        border-radius: 12px;
        box-shadow: 0 15px 35px rgba(0,0,0,0.2);
        text-align: center;
    }
    .login-header i {
        color: #3498db;
        margin-bottom: 15px;
    }
            .login-header h2 {
```

Screenshot of its css

3.2.2 Sales Page Development

The sales page was developed to allow users to record and manage sales transactions within the system. This page enables the user to select a product, enter the quantity sold and automatically calculate the total amount sold. The information is then stored in the database for future reference and reporting.



Sales page

```
/head>

body>

div class="terminal-card">

header>
h2><i class="fas fa-cash-register"></i> SALES / CHECKOUT</h2>
/header>

main>

div class="input-group">
label>Product:</label>

select id="productSelect" onchange="updatePrice()">

option value="" disabled selected>Select Product</option>

?php while($row = mysqli_fetch_assoc($products)){ ?>

option
alue="<?php echo $row['id']; ?>"
ata-price="<?php echo $row['price']; ?>"
ata-stock="<?php echo $row['quantity']; ?>">

?php echo $row['name']; ?> (Stock: <?php echo $row['quantity']; ?>)
```

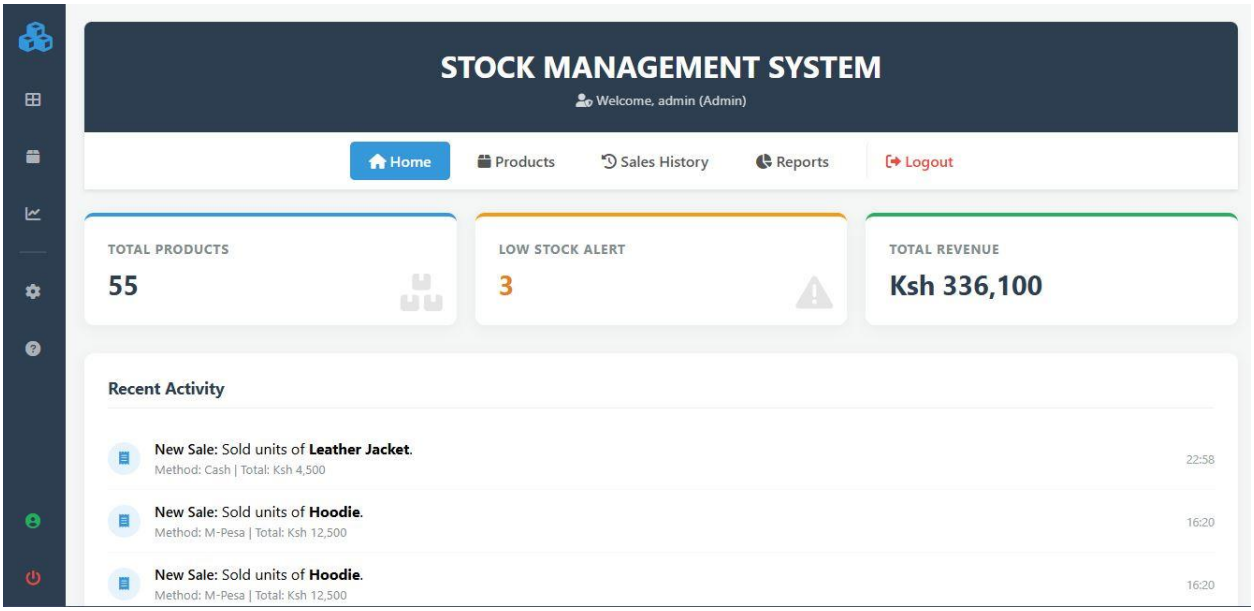
Screenshot of its html

```
<style>

body{
background:#000;
font-family:monospace;
display:flex;
justify-content:center;
align-items:center;
height:100vh;
color:#00ff66;
}
.terminal-card{
width:420px;
border:2px solid #00ff66;
padding:20px;
background:#000;
box-shadow:0 0 20px #00ff66;
}
header h2{
text-align:center;
margin-bottom:15px;
border-bottom:1px dashed #00ff66;
padding-bottom:10px;
}
.input-group{
margin:10px 0;
display:flex;
justify-content:space-between;
align-items:center;
}
label{
```

Screenshot of its css

3.2.4 Admin Dashboard Development

This is where the admin views the shop stocks and records, shows alerts of low stocks and also captures the revenue and sales history



Admin dashboard

```html
<?php endif; ?>
    <aside class="side-bar">
        <div class="sidebar-links" style="flex: 1; display: flex; flex-direction: column;">

            <hr style="width: 40%; align-self: center; border: 0.5px solid rgba(255,255,255,0.1); margin: 10px 0;">

            <a class="side-item" title="Settings" onclick="showTab('settings', this)"><i class="fas fa-cog"></i></a>
            <a class="side-item" title="Help" onclick="showTab('help', this)"><i class="fas fa-question-circle"></i></a>
        </div>

        <div class="sidebar-bottom" style="display: flex; flex-direction: column;">
            <a class="side-item" title="Profile" onclick="showTab('profile', this)" style="color: var(--success-green);"><i class="fas fa-user-ci
            <a class="side-item" title="Logout" href="logout.php" style="color: var(--danger-red);"><i class="fas fa-power-off"></i></a>
        </div>
    </aside>

    <div class="dashboard-container">
        <header class="main-header">
    <h1>STOCK MANAGEMENT SYSTEM</h1>
    <p style="font-size: 0.9rem; margin-top: 5px; opacity: 0.8;">
        <i class="fas fa-user-shield"></i> Welcome, <?php echo htmlspecialchars($_SESSION['username']); ?> (Admin)
    </p>
</header>
        <nav class="nav-bar">
            <a href="javascript:void(0)" class="nav-item active" onclick="showTab('home', this)"><i class="fas fa-home"></i> Home</a>
            <a href="javascript:void(0)" class="nav-item" onclick="showTab('products', this)"><i class="fas fa-box"></i> Products</a>
            <a href="javascript:void(0)" class="nav-item" onclick="showTab('sales', this)"><i class="fas fa-clock-rotate-left"></i> Sales History
            <a href="javascript:void(0)" class="nav-item" onclick="showTab('reports', this)"><i class="fas fa-chart-pie"></i> Reports</a>
            <a href="logout.php" class="nav-item" style="color: var(--danger-red); border-left: 1px solid #eee; margin-left: 15px;" onclick="re
                <i class="fas fa-sign-out-alt"></i> Logout
            </a>
```

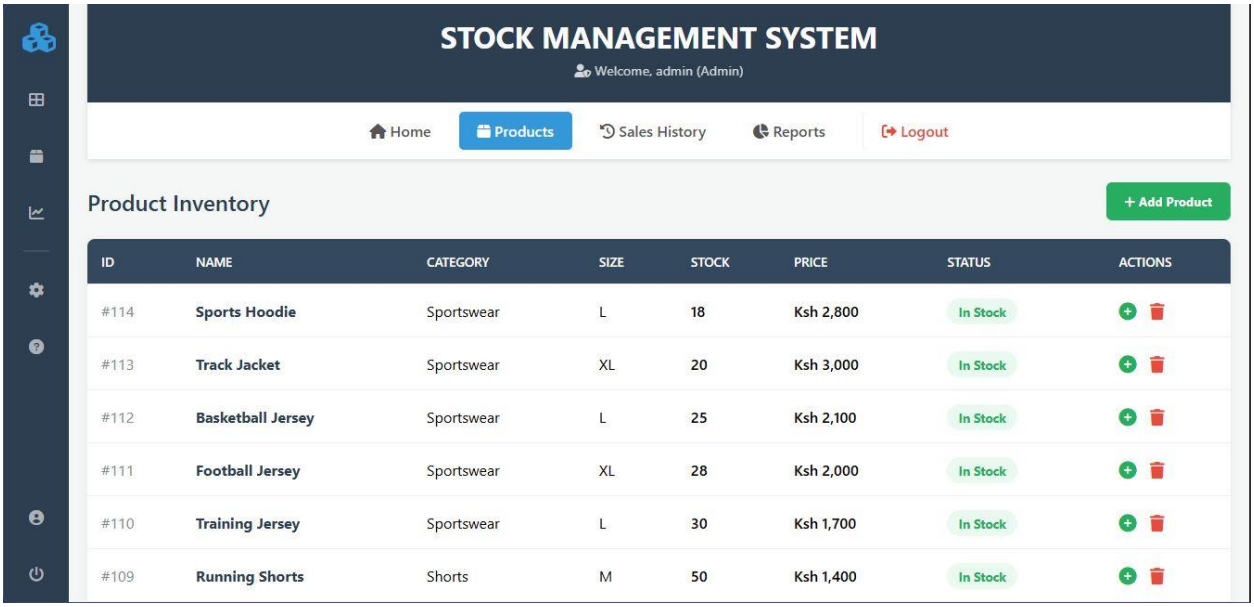
Screenshot of its html

```html
<head>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css">

    <style>
        :root {
            --primary-dark: #2c3e50;
            --sidebar-width: 70px;
            --bg-light: #f4f7f6;
            --accent-blue: #3498db;
            --success-green: #27ae60;
            --warning-orange: #f39c12;
            --danger-red: #e74c3c;
            --white: #ffffff;
        }

        * { margin: 0; padding: 0; box-sizing: border-box; font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; }
        body { background-color: var(--bg-light); display: flex; min-height: 100vh; }

        /* SIDEBAR */
        .side-bar {
            width: var(--sidebar-width);
            background: var(--primary-dark);
            height: 100vh;
            position: fixed;
            display: flex;
            flex-direction: column;
            color: white;
            z-index: 100;
        }
        .side-item {
            padding: 20px;
```

Screenshot of its html

3.2.5 Stock Management Page Development

The stock management page is a crucial part of the system as it allows the administrator to manage all the products available for sale. This includes adding new products, viewing existing products, editing product information and deleting products that are no longer needed.



Stock mgt

```
C: > xampp > htdocs > stock_system > admin.php
290     </header>
347         <div id="products" class="tab-content" style="display:none;">
348             <div style="display:flex; justify-content: space-between; align-items: center; margin-bottom: 20px;">
349                 <h2 style="color: var(--primary-dark); font-weight: 600;">Product Inventory</h2>
350                 <button class="add-btn" onclick="openModal('productModal')"><i class="fas fa-plus"></i> Add Product</button>
351             </div>
352             <table class="inventory-table">
353                 <thead>
354                     <tr>
355                         <th>ID</th>
356                         <th>Name</th>
357                         <th>Category</th>
358                         <th>Size</th>
359                         <th>Stock</th>
360                         <th>Price</th>
361                         <th>Status</th>
362                         <th>Actions</th>
363                     </tr>
364                 </thead>
365                 <tbody>
366                     <?php if($products_query && mysqli_num_rows($products_query) > 0): ?>
367                         <?php while($row = mysqli_fetch_assoc($products_query)):
368                             $stock = $row['quantity'];
369                             if($stock <= 0) {
370                                 $status = '<span class="badge badge-danger">Out of Stock</span>';
371                             } elseif($stock < 10) {
372                                 $status = '<span class="badge badge-warning">Low Stock</span>';
373                             } else {
374                                 $status = '<span class="badge badge-success">In Stock</span>';
375                             }
```

Screenshot of its html

```
<style>
    .badge { padding: 5px 12px; border-radius: 20px; font-size: 0.85rem; font-weight: bold; }
    .badge-success { background: #eafaf1; color: #27ae60; }
    .badge-warning { background: #fff3e0; color: #f39c12; }
    .badge-danger { background: #fdeaea; color: #e74c3c; }

    /* TABLES */
    .inventory-table {
        width: 100%; background: white; border-collapse: collapse; border-radius: 8px;
        box-shadow: 0 2px 10px rgba(0,0,0,0.05); margin-top: 20px; overflow: hidden;
    }
    .inventory-table thead tr { background-color: #34495e; color: white; }
    .inventory-table th { padding: 15px; text-align: left; font-weight: 600; text-transform: uppercase; font-size: 0.85rem; }
    .inventory-table td { padding: 15px; border-top: 1px solid #eee; vertical-align: middle; }
    .inventory-table tr:hover { background-color: #fcfcfc; }

    .activity-section { background: white; padding: 25px; border-radius: 10px; box-shadow: 0 4px 15px rgba(0,0,0,0.05); }
    .activity-item { display: flex; align-items: center; padding: 12px 0; border-bottom: 1px solid #f9f9f9; }
    .activity-icon { width: 35px; height: 35px; background: #e8f4fd; color: var(--accent-blue); border-radius: 50%; display: flex; align-it
```

Screenshot of its css

3.3 Logic Development

3.3.1 Login Validation Code

The login validation process ensures that only authorized users can access the system and when a user enters their username and password in the login form, the system first checks whether the provided details exist in the database. If the credentials match the stored records, the user is granted access to the system dashboard. If the credentials are incorrect, the system displays an error message.

```php
<?php
session_start();
include('db.php');

$error = "";

// Check if form is submitted
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = mysqli_real_escape_string($conn, $_POST['username']);
    $password = $_POST['password']; // Don't escape the password yet, we need it raw for verify
    $role = mysqli_real_escape_string($conn, $_POST['role']);

    // 1. Search for the user by username and role
    $query = "SELECT * FROM users WHERE username='$username' AND role='$role'";
    $result = mysqli_query($conn, $query);

    if (mysqli_num_rows($result) == 1) {
        $user = mysqli_fetch_assoc($result);

        // 2. USE PASSWORD_VERIFY to check against the hashed password in the DB
        if (password_verify($password, $user['password'])) {

            $_SESSION['username'] = $user['username'];
            $_SESSION['role'] = $user['role'];

            // Redirection Logic
            if ($user['role'] == 'admin') {
                header("Location: admin.php");
            } else {
                header("Location: sales.php");
            }
```

3.3.2 Admin Dashboard Validation Code

The admin dashboard validation code ensures that only authorized administrators can access the admin section of the system. When a user logs into the system, their role is stored in a session variable and the system then checks this session variable before allowing access to the admin dashboard page.

```php
<?php endif; ?>
    <aside class="side-bar">
        <div class="sidebar-links" style="flex: 1; display: flex; flex-direction: column;">

            <hr style="width: 40%; align-self: center; border: 0.5px solid rgba(255,255,255,0.1); margin: 10px 0;">

            <a class="side-item" title="Settings" onclick="showTab('settings', this)"><i class="fas fa-cog"></i></a>
            <a class="side-item" title="Help" onclick="showTab('help', this)"><i class="fas fa-question-circle"></i></a>
        </div>

        <div class="sidebar-bottom" style="display: flex; flex-direction: column;">
            <a class="side-item" title="Profile" onclick="showTab('profile', this)" style="color: var(--success-green);"><i class="fas fa-user-ci
            <a class="side-item" title="Logout" href="logout.php" style="color: var(--danger-red);"><i class="fas fa-power-off"></i></a>
        </div>
    </aside>

    <div class="dashboard-container">
        <header class="main-header">
    <h1>STOCK MANAGEMENT SYSTEM</h1>
    <p style="font-size: 0.9rem; margin-top: 5px; opacity: 0.8;">
        <i class="fas fa-user-shield"></i> Welcome, <?php echo htmlspecialchars($_SESSION['username']); ?> (Admin)
    </p>
</header>
        <nav class="nav-bar">
            <a href="javascript:void(0)" class="nav-item active" onclick="showTab('home', this)"><i class="fas fa-home"></i> Home</a>
            <a href="javascript:void(0)" class="nav-item" onclick="showTab('products', this)"><i class="fas fa-box"></i> Products</a>
            <a href="javascript:void(0)" class="nav-item" onclick="showTab('sales', this)"><i class="fas fa-clock-rotate-left"></i> Sales History
            <a href="javascript:void(0)" class="nav-item" onclick="showTab('reports', this)"><i class="fas fa-chart-pie"></i> Reports</a>
            <a href="logout.php" class="nav-item" style="color: var(--danger-red); border-left: 1px solid #eee; margin-left: 15px;" onclick="re
                <i class="fas fa-sign-out-alt"></i> Logout
            </a>
```

3.3.3 Stock Management Validation Code

In this section, the reader expects to see how the system was tested and validated to ensure accurate stock management and proper handling of sales transactions. It demonstrates that the system prevents overselling, updates inventory automatically and maintains reliable records of all transactions.

```php
// Total Products
$total_products = 0;
$query_total = mysqli_query($conn, "SELECT COUNT(*) as total FROM products");
if($query_total){
    $total_products = mysqli_fetch_assoc($query_total)['total'] ?? 0;
}

// Low Stock
$low_stock = 0;
$query_low = mysqli_query($conn, "SELECT COUNT(*) as low FROM products WHERE quantity < 10");
if($query_low){
    $low_stock = mysqli_fetch_assoc($query_low)['low'] ?? 0;
}

// Default Values
$total_revenue = 0;
$sales_history_query = false;
$recent_activity_query = false;

// ==============================
// CHECK IF SALES TABLE EXISTS
// ==============================

$val_table = mysqli_query($conn, "SHOW TABLES LIKE 'sales'");

if($val_table && mysqli_num_rows($val_table) > 0){
```

3.3.4 Sales Validation Code

The sales page is a designed to record and manage product sales while keeping track of inventory. It ensures that sales transactions are accurate, validated and stored for future reference.

```
/head>

body>

div class="terminal-card">

header>
h2><i class="fas fa-cash-register"></i> SALES / CHECKOUT</h2>
/header>

main>

div class="input-group">
label>Product:</label>

select id="productSelect" onchange="updatePrice()">

option value="" disabled selected>Select Product</option>

?php while($row = mysqli_fetch_assoc($products)){ ?>

option
alue="<?php echo $row['id']; ?>"
ata-price="<?php echo $row['price']; ?>"
ata-stock="<?php echo $row['quantity']; ?>">

?php echo $row['name']; ?> (Stock: <?php echo $row['quantity']; ?>)
```

3.3.5 Database Design

The Stock Management System uses a MySQL database to store and manage all system data. The database ensures that product information, user accounts and sales transactions are stored securely and can be retrieved whenever required. The system database consists of three main tables: Users, Products and Sales.

These tables are connected to allow smooth system operation such as login authentication, stock updates and sales recording.

Table 3.3.5.1 Users Table

This table stores the login credentials and roles of users who are allowed to access the system.

Field Name	Data Type	Description
user_id	INT (Primary Key)	Unique identifier for each user
username	VARCHAR(50)	Username used during login

Field Name	Data Type	Description
password	VARCHAR(255)	Encrypted password for authentication
role	VARCHAR(20)	User role (Admin or Staff)

Table 3.3.5.1 Users Table Structure

Table 3.3.5.2 Products Table

This table stores information about all products available in the shop inventory.

Field Name	Data Type	Description
product_id	INT (Primary Key)	Unique identifier for each product
product_name	VARCHAR(100)	Name of the product
category	VARCHAR(50)	Type or category of the product
price	DECIMAL(10,2)	Price of the product
quantity	INT	Current stock quantity
date_added	DATE	Date when the product was added

Table 3.3.5.2 Products Table Structure

Table 3.3.5.3 Sales Table

This table stores all sales transactions recorded in the system.

Field Name	Data Type	Description
sale_id	INT (Primary Key)	Unique identifier for each sale
product_id	INT (Foreign Key)	ID of the product sold
quantity_sold	INT	Number of items sold
total_price	DECIMAL(10,2)	Total amount for the sale

Field Name	Data Type	Description
sale_date	DATE	Date when the sale occurred

Table 3.3.5.3 Sales Table Structure

These database tables work together to ensure efficient system operation. The Users table manages system authentication and access control. The Products table stores inventory information including product names, prices and quantities. The Sales table records all sales transactions and automatically updates stock levels when a product is sold. This database design ensures that the system maintains accurate records and supports reliable inventory management.

3.4 Testing

The sales page and stock management modules were thoroughly tested to ensure they functioned correctly and reliably. The testing focused on verifying that sales transactions were recorded accurately, stock levels were updated automatically and the system prevented invalid sales. Functional testing confirmed that users could select a product, enter the quantity, calculate the total price and record the sale successfully. Validation testing ensured that the quantity entered could not exceed available stock and that negative or zero values were rejected, with appropriate error messages displayed. Database testing confirmed that each valid sale was stored in the sales table and that the corresponding product stock was updated in the products table. Edge cases, such as selling exactly the remaining stock or attempting to sell more than the available stock, were also tested to confirm correct system behavior. The results showed that the system accurately prevented overselling, calculated totals correctly and maintained reliable records of all transactions demonstrating that the Sales and Stock Management modules are robust, user-friendly and effective for managing inventory and sales operations.

3.5 Deployment

The deployment of the Stock Management System was carried out using both a local development environment and an online web hosting platform. During the development phase, the system was first tested using the XAMPP server environment which provides the necessary tools for running web-based applications. XAMPP includes the PHP programming environment and MySQL database which were required for developing and testing the system.

The system files were initially stored in the **htdocs** directory within the XAMPP installation folder, allowing the application to be accessed locally through a web browser using the localhost server. The database required for the system was created using phpMyAdmin, where tables such as users, products, and sales were created to store system data.

After successful testing in the local environment, the system was deployed online using the InfinityFree web hosting service. The system files were uploaded to the hosting server using the File Manager provided in the InfinityFree control panel. The database was also recreated in the hosting environment and connected to the system using updated database configuration settings.

The deployed system can now be accessed through the following web address:

https://veshka.ct.ws

Through this website, users can access the login page, manage products, record sales transactions, and monitor stock levels from any location using a web browser.

After deployment, the system was tested again to ensure that all functionalities such as login validation, product management, sales recording and database connectivity were working correctly in the live environment. The results confirmed that the system operates smoothly and meets the intended requirements for managing inventory and sales operations.

CHAPTER FOUR: CONCLUSION AND RECOMMENDATION

4.1 Conclusion

This chapter explains how the system was developed, tested and executed. It describes the logic used in different parts of the system such as login validation, user registration and dashboard access.

Testing was also carried out to ensure that all features of the system work correctly. Errors found during testing were corrected to improve the performance and reliability of the system.

Finally, the system was successfully deployed using the XAMPP server environment allowing users to access and use the system through a web browser. Overall the system functions as intended and is ready for use.

4.2 Recommendation

Based on the development and testing of the system, several recommendations can be made to improve the system in the future.

First the system can be improved by adding more advanced security features such as password encryption and stronger user authentication to protect user data.

Secondly the system can be enhanced by adding more functionalities such as automated reports, notifications and improved search features to make it more efficient for users.

Third the system could be deployed online so that users can access it from different locations instead of only using it on a local server.

Finally, the system was successfully deployed using an online web hosting platform (InfinityFree) allowing users to access the system through a web browser from different locations.

REFERENCES

- Welling, L., & Thomson, L. (2017). *PHP and MySQL Web Development* (5th ed.). Addison-Wesley.
  This book covers development of dynamic web applications using PHP and MySQL, including database interaction, form handling, and validation.

- Ullman, L. (2018). *PHP for the Web: Visual Quick Start Guide* (5th ed.). Peach pit press. provides practical guidance on building php web applications, including handling forms, sessions, and database integration.

- W3Schools. *PHP Form Validation*. Retrieved March 18, 2026, from https://www.w3schools.com/php/php_form_validation.asp

- MDN Web Docs. (2024). *HTML forms guide*. Retrieved from https://developer.mozilla.org/en-US/docs/Learn/Forms